

1.Web Gauntlet 1-3

02 July 2025 14:47

Web Gauntlet 1:

- On trying basic injections with username admin we figure out the username admin is valid.
- The filter.php subdomain show which all words or symbols are filtered so they wont pass to sql
- Hence with reference to them we can solve the first three simply by adding a colon ' to the end of admin and adding a semicolon ; as the semi-colon would terminate that line before it goes to password and hence the first three can be solved in the same way.
- The 4th one however blocks the admin word, so instead of putting in admin directly we concatenate it using || operator to concatenate and the same goes for 5th as they just block UNION which is another format of concatenating and on solving we get the flag:
picoCTF{y0u_m4d3_1t_cab35b843fdd6bd889f76566c6279114}

Web Gauntlet 2:

- This filters more than earlier but this can easily be solved by IS NOT method and by concatenating the username which is admin by using the || operator like in the last question. How the IS NOT method works is by equating the to {" IS NOT '(somevalue)"} what this gives out is TRUE as " is not equal to some other value which could be anything making the value of password TRUE hence giving it a go to open and give the flag:
picoCTF{0n3_m0r3_t1m3_fc0f841ee8e0d3e1f479f1a01a617ebb}

Web Gauntlet 3:

- This can also be solved the same way as above using the IS NOT method.
Flag:
picoCTF{k3ep_1t_sh0rt_2a78ea34c84da0bf585ada4cb9a6f8fb}

2. IRISH REPO 1-3

03 July 2025 15:18

Irish Repo 1:

- This is fairly simple , and like last time we jus need to enter the username which on first guess we realise is admin, and we just need to terminate the command right after that which gives back the flag, the interesting thing is there is a debugger which give feedback if you put in a value which isn't 0 , it gives back the command it sends to the database.

Flag:picoCTF{s0m3_SQL_c218b685}

Irish Repo 2:

- This also can be solved by the same method above.

Flag: Your flag is: picoCTF{m0R3_SQL_plz_fa983901}

Irish Repo 3:

- This is a bit more complex as there is no username section so we have to enter the password in directly , so here we can use the IS NOT trick to make the password equate to TRUE letting us pass through, but we can see that login has failed , but here the debugger comes in handy again as we can see the input we put in has been modified to something else which looks suspiciously similar to something we know, at first look I tried ROT-13 and it turned out to be the ROT-13 cipher. So we need to enter the same command but in ROT-13 text as its being converted into its ROT-13 form, again on passing we finally get the flag.

Flag: picoCTF{3v3n_m0r3_SQL_7f5767f6}

(VIDEO SOLUTIONS FOR ALL OF THE QUESTIONS ABOVE IS ATTACHED)

3. JaWT Scrathpad

07 July 2025 16:18

- We are given two hints with the question , one pointing to cookies and the other to JWT.
- On opening the website we find two hyperlinks which lead to a JWT decoder and one to the github page of JOHN THE RIPPER which are clues to what might be used as tools in the challenge.
- And a hint is given that admin has a special pad so we can assume that entering admin will be unlock the flag , but on entering we're denied permission , as we don't have the permission to access it.
- On opening the console , we find a cookie with the name JWT , so we run it through the JWT decoder and we find out that it's stores the info , but we have to find the valid signature for the same. And we can see the algorithm is HS-256 for the signature, as we need a valid signed jwt for the website to accept it else it'd end up in error, as the header and payload are simple encoded (base64) , but the signature is the secret key that makes it special for authentication.
- So we put it in a file and run it through john the ripper to crack , for JWT with HS256 algorithms, they require a secret key (signature) which is a HMAC-SHA256(HS256) digest, and so we run it with the format as HMAC-SHA256, and we get the required key.
- Now running it through a decoder , we find out its valid.
- Then copying the same json and adding the key we get the required jwt , if we paste it in the cookie again , and then refresh the page , we get the flag in the scratch pad.

Flag: picoCTF{jawt_was_just_what_you_thought_44c752f5}

4.Secrets

30 June 2025 23:13

- First launch caido and set the proxy as usual.
- Then open the website
- Turn on intercept to intercept requests.
- Then on experimenting we figure out that just getting /secret/ (GET /secret/) we get something suspicious back where the site says we're getting closer.
- Then in the response we find a hyperlink linking to /hidden , we get no luck on GET /hidden but on GET /secret/hidden we find a login page (or you could jus do url/secret/hidden/) .
- By now we've turned off intercept mode so that requests are still forwarded and on observing sitemaps , we find out a new folder with a suspicious name popping up (/superhidden) (or by observing the response from passing /secret/hidden/ we can see another hyperlink connecting to /superhidden) ,which on passing we get a message saying we found the flag .
- Now if you check the response from the request sent to open superhidden we find the flag in the script.
(ViDEO SOLUTION ATTACHED)
Flag:picoCTF{succ3ss_@h3n1c@10n_39849bcf}

5. Client Side Again

04 July 2025 16:04

- Opening the site we see that its jus a box asking for the password
- On further inspecting by opening the console and checking the javascript code we find out the code is essentially unreadable
- But As mentioned in the hints it is obfuscated , so we can Un-obfuscate it by using something like the deobfuscate.io , and then looking at the code we can just focus on the verify part , and the top array.
- The top array consists of some items suspiciously similar to the flag , so on checking the verify part of the code we can see that its checking the input against the parts of the array , and we can see the split variable is assigned to 0x4. From this by calling each part it is equated to in the console and essentially replacing the part of the input it is equated we can reverse engineer the password giving us the final flag being:
picoCTF{not_this_again_337115}
(Video Solution attached)

7.IntroToBurp

29 June 2025 22:48

- Launch the instance
- Open the website and enter the details.
- When the 2fa page opens up , turn on intercept.
- Then as mentioned in the hints if you hinder the response it might not handle it well, so make some error on purpose for example change the otp variable to something else.
- The server gives back the flag as response.
(Video Solution attached)
Flag: picoCTF{#OTP_Bypvss_SuCc3\$S_6bffd21}